WEEK 9: SALES AND CUSTOMER ANALYTICS SYSTEM

TRAVEL BOOKING MANAGEMENT SYSTEM

AIM

To analyze the sales and booking data of the Travel Booking Management System using SQL aggregate functions and generate reports to understand customer booking behaviour, booking amounts, payment trends, and travel package information.

INTRODUCTION

The Travel Booking Management System is designed to manage customer details, travel packages, bookings, payments, and customer reviews. Sales and customer analytics help to understand the booking behaviour of customers and evaluate the financial performance of the system.

SQL aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX() are used to perform calculations on multiple records and generate summarized results. These functions help identify total bookings, total booking amounts, average booking values, highest and lowest booking amounts, frequent customers, and payment trends.

In this report, SQL operations are used to analyze Travel Booking Management System data and generate business analytics reports.

1. APPLY SQL AGGREGATE FUNCTIONS

Aggregate functions perform calculations on multiple records and return a single summarized value.

Function	Purpose
COUNT()	Counts the number of records
SUM()	Calculates the total booking amount
AVG()	Finds the average booking amount
MIN()	Finds the lowest booking amount
MAX()	Finds the highest booking amount

These functions are used in the following sections to analyze booking, customer, package, and payment information.

2. GENERATE TOTAL SALES REPORTS

1. Total number of bookings placed

The total number of bookings placed by customers in the Travel Booking Management System is calculated using the COUNT() aggregate function. It counts all the records available in the BOOKING table.

```
SQL> select count(*) as total_bookings from booking;

TOTAL_BOOKINGS
--------------
             8
```

2. Total revenue generated from bookings

The total revenue generated from bookings is calculated using the SUM() function. It adds the total amount of every booking recorded in the BOOKING table.

```
SQL> select sum(total_amount) as total_revenue from booking;

TOTAL_REVENUE
-------------
       335000
```

3. Average booking value

The average booking value is calculated using the AVG() function. It finds the average of all booking amounts recorded in the BOOKING table.

```
SQL> select avg(total_amount) as average_booking_amount from booking;

AVERAGE_BOOKING_AMOUNT
----------------------
                 41875
```

4. Highest and lowest booking amount

The MAX() function identifies the highest booking amount, while the MIN() function identifies the lowest booking amount recorded in the system.

```
SQL> select min(total_amount) as lowest_amount, max(total_amount) as highest
_amount from booking;

LOWEST_AMOUNT HIGHEST_AMOUNT
------------- --------------
        24000          65000
```

5. Total booking amount for a specific period

The total booking amount generated between a specified date range is calculated by filtering bookings based on the booking date. The SUM() function is used to calculate the total amount for the selected period.

```
SQL> select sum(total_amount) as period_revenue from booking where booking_d
ate between date '2026-09-20' and date '2026-09-24';

PERIOD_REVENUE
--------------
```

3. CUSTOMER PURCHASE ANALYSIS

1. Total number of bookings placed by each customer

The total number of bookings placed by each customer is calculated using the COUNT() function. The CUSTOMER and BOOKING tables are joined to display the booking count for every customer. Customers who have not placed any bookings are also included.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS
  5  FROM CUSTOMER C
  6  LEFT JOIN BOOKING B
  7  ON C.CUSTOMER_ID = B.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME;

CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_BOOKING
S
-------------------- -------------------- -------------------- -------------
-
C005                 Karthik              Mohan
1
C008                 Ananya               Krishnan
1
C001                 Arun                 Kumar
1
C004                 Divya                Suresh
1
C002                 Priya                Sharma
1
C003                 Rahul                Raj
1
C006                 Sneha                Ravi
1
C007                 Vishnu               Kumar
1
C010                 Meena                Devi
0
C009                 Naveen               Raj
0

10 rows selected.
```

2. Total amount spent by each customer

The total amount spent by each customer is calculated using the SUM() function. It adds the booking amounts for each customer. The NVL() function replaces NULL values with zero for customers who have not made any bookings.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         NVL(SUM(B.TOTAL_AMOUNT), 0) AS TOTAL_SPENT
  5  FROM CUSTOMER C
  6  LEFT JOIN BOOKING B
  7  ON C.CUSTOMER_ID = B.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME;

CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_SPENT
-------------------- -------------------- -------------------- -----------
C005                 Karthik              Mohan                      25000
C008                 Ananya               Krishnan                   55000
C001                 Arun                 Kumar                      24000
C004                 Divya                Suresh                     65000
C002                 Priya                Sharma                     36000
C003                 Rahul                Raj                        55000
C006                 Sneha                Ravi                       30000
C007                 Vishnu               Kumar                      45000
C010                 Meena                Devi                           0
C009                 Naveen               Raj                            0

10 rows selected.
```

3. Average spending per customer

The average spending of each customer is calculated using the AVG() function. It finds the average booking amount for every customer. Customers without bookings are displayed with zero.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         NVL(AVG(B.TOTAL_AMOUNT), 0) AS AVERAGE_SPENDING
  5  FROM CUSTOMER C
  6  LEFT JOIN BOOKING B
  7  ON C.CUSTOMER_ID = B.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME;

CUSTOMER_ID          FIRST_NAME           LAST_NAME            AVERAGE_SPEND
ING
-------------------- -------------------- -------------------- -------------
---
C005                 Karthik              Mohan                         25
000
C008                 Ananya               Krishnan                      55
000
C001                 Arun                 Kumar                         24
000
C004                 Divya                Suresh                        65
000
C002                 Priya                Sharma                        36
000
C003                 Rahul                Raj                           55
000
C006                 Sneha                Ravi                          30
000
C007                 Vishnu               Kumar                         45
000
C010                 Meena                Devi
  0
C009                 Naveen               Raj
  0

10 rows selected.
```

4. Customers with maximum purchase amount

This analysis identifies the customer or customers who have spent the highest total amount on bookings. The total spending of each customer is calculated and compared to identify the maximum booking amount.

```
SQL> SELECT CUSTOMER_ID,
  2         FIRST_NAME,
  3         LAST_NAME,
  4         TOTAL_SPENT
  5  FROM (
  6      SELECT C.CUSTOMER_ID,
  7             C.FIRST_NAME,
  8             C.LAST_NAME,
  9             SUM(B.TOTAL_AMOUNT) AS TOTAL_SPENT
 10      FROM CUSTOMER C
 11      JOIN BOOKING B
 12      ON C.CUSTOMER_ID = B.CUSTOMER_ID
 13      GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
 14  )
 15  WHERE TOTAL_SPENT = (
 16      SELECT MAX(TOTAL_SPENT)
 17      FROM (
 18          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
 19          FROM BOOKING
 20          GROUP BY CUSTOMER_ID
 21      )
 22  );

CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_SPENT
-------------------- -------------------- -------------------- -----------
C004                 Divya                Suresh                     65000
```

5. Customers with fewer purchases

This analysis identifies customers who have placed one or fewer bookings. The number of bookings placed by each customer is counted to identify customers with fewer purchases.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS
  5  FROM CUSTOMER C
  6  LEFT JOIN BOOKING B
  7  ON C.CUSTOMER_ID = B.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
  9  HAVING COUNT(B.BOOKING_ID) <= 1;

CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_BOOKING
S
-------------------- -------------------- -------------------- -------------
-
C005                 Karthik              Mohan
1
C008                 Ananya               Krishnan
1
C001                 Arun                 Kumar
1
C004                 Divya                Suresh
1
C002                 Priya                Sharma
1
C003                 Rahul                Raj
1
C006                 Sneha                Ravi
1
C007                 Vishnu               Kumar
1
C010                 Meena                Devi
0
C009                 Naveen               Raj
0

10 rows selected.
```

4. FIND TOP CUSTOMERS BASED ON PURCHASE AMOUNT

1. Top 5 customers based on total spending

This analysis displays the top five customers based on their total spending. The total booking amount of each customer is calculated and arranged in descending order to identify the highest-spending customers.

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CUSTOMER_ID,
  4             C.FIRST_NAME,
  5             C.LAST_NAME,
  6             SUM(B.TOTAL_AMOUNT) AS TOTAL_SPENT
  7      FROM CUSTOMER C
  8      JOIN BOOKING B
  9      ON C.CUSTOMER_ID = B.CUSTOMER_ID
 10      GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
 11      ORDER BY TOTAL_SPENT DESC
 12  )
 13  WHERE ROWNUM <= 5;

CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_SPENT
-------------------- -------------------- -------------------- -----------
C004                 Divya                Suresh                     65000
C008                 Ananya               Krishnan                   55000
C003                 Rahul                Raj                        55000
C007                 Vishnu               Kumar                      45000
C002                 Priya                Sharma                     36000
```

2. Customers with maximum number of bookings

This analysis identifies the customer or customers who have placed the maximum number of bookings. The total number of bookings placed by each customer is calculated and compared.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS
  5  FROM CUSTOMER C
  6  JOIN BOOKING B
  7  ON C.CUSTOMER_ID = B.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
  9  HAVING COUNT(B.BOOKING_ID) = (
 10      SELECT MAX(BOOKING_COUNT)
 11      FROM (
 12          SELECT COUNT(*) AS BOOKING_COUNT
 13          FROM BOOKING
 14          GROUP BY CUSTOMER_ID
 15      )
 16  );

CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_BOOKING
S
-------------------- -------------------- -------------------- -------------
-
C005                 Karthik              Mohan
1
C008                 Ananya               Krishnan
1
C001                 Arun                 Kumar
1
C004                 Divya                Suresh
1
C002                 Priya                Sharma
1
C003                 Rahul                Raj
1
C006                 Sneha                Ravi
1
C007                 Vishnu               Kumar
1

8 rows selected.
```

3. Frequent customers

This analysis identifies frequent customers who have placed more than one booking in the Travel Booking Management System. The number of bookings placed by each customer is calculated to identify customers with repeated bookings.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS
  5  FROM CUSTOMER C
  6  JOIN BOOKING B
  7  ON C.CUSTOMER_ID = B.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
  9  HAVING COUNT(B.BOOKING_ID) > 1
 10  ORDER BY TOTAL_BOOKINGS DESC;

no rows selected
```

4. High-value customers

This analysis identifies high-value customers whose total booking amount is ₹4,000 or more. The total spending of each customer is calculated to identify customers who contribute more to the overall booking amount.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         SUM(B.TOTAL_AMOUNT) AS TOTAL_SPENT
  5  FROM CUSTOMER C
  6  JOIN BOOKING B
  7  ON C.CUSTOMER_ID = B.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
  9  HAVING SUM(B.TOTAL_AMOUNT) >= 4000
 10  ORDER BY TOTAL_SPENT DESC;

CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_SPENT
-------------------- -------------------- -------------------- -----------
C004                 Divya                Suresh                     65000
C008                 Ananya               Krishnan                   55000
C003                 Rahul                Raj                        55000
C007                 Vishnu               Kumar                      45000
C002                 Priya                Sharma                     36000
C006                 Sneha                Ravi                       30000
C005                 Karthik              Mohan                      25000
C001                 Arun                 Kumar                      24000

8 rows selected.
```

5. ANALYZE TRAVEL PACKAGE PERFORMANCE

The TRAVELPACKAGE table contains information about travel packages, including package names, prices, categories, and available slots. Package price and availability analysis help understand package information and identify packages that require attention.

1. Packages arranged by highest price

This analysis displays travel packages according to their prices. The packages are arranged in descending order to identify the highest-priced travel packages.

```
SQL> SELECT PACKAGE_ID,
  2         PACKAGE_NAME,
  3         PRICE
  4  FROM TRAVELPACKAGE
  5  ORDER BY PRICE DESC;

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
       104
Singapore Tour Package
     65000

       103
Dubai Holiday Package
     55000

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------

       108
Kashmir Honeymoon Package
     45000

       107
Goa Honeymoon Package

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
     30000

       110
```

```
Rajasthan Family Tour
     28000

       105

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
Manali Adventure Trip
     25000

       106
Rishikesh Adventure Tour
     22000


PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
       109
Mysore Family Package
     15000

       112
Varanasi Spiritual Tour
     14000

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------

       101
Ooty Holiday Package
     13000
```

```
       111
Tirupati Pilgrimage Tour

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
      9000


11 rows selected.
```

2. Packages with the lowest available slots

This analysis identifies travel packages with the lowest number of available slots. It helps understand package availability and identify packages with limited remaining slots.

```
SQL> SELECT PACKAGE_ID,
  2         PACKAGE_NAME,
  3         AVAILABLE_SLOTS
  4  FROM TRAVELPACKAGE
  5  ORDER BY AVAILABLE_SLOTS ASC;

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
AVAILABLE_SLOTS
---------------
       108
Kashmir Honeymoon Package
              8

       103
Dubai Holiday Package
             10

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
AVAILABLE_SLOTS
---------------

       107
Goa Honeymoon Package
             10

       104
Singapore Tour Package

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
AVAILABLE_SLOTS
---------------
             12

       106
```

```
Rishikesh Adventure Tour
             14

       110

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
AVAILABLE_SLOTS
---------------
Rajasthan Family Tour
             16

       105
Manali Adventure Trip
             18


PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
AVAILABLE_SLOTS
---------------
       112
Varanasi Spiritual Tour
             20

       101
Ooty Holiday Package
             20

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
AVAILABLE_SLOTS
---------------

       109
Mysore Family Package
             20
```

```
       111
Tirupati Pilgrimage Tour

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
AVAILABLE_SLOTS
---------------
             25


11 rows selected.
```

3. Packages with zero available slots

This analysis identifies travel packages that have no available slots. It helps determine which packages are currently unavailable for new bookings.

```
SQL> SELECT PACKAGE_ID,
  2         PACKAGE_NAME,
  3         PRICE,
  4         AVAILABLE_SLOTS
  5  FROM TRAVELPACKAGE
  6  WHERE AVAILABLE_SLOTS = 0;

no rows selected
```

4. Packages requiring promotion or attention

This analysis identifies packages with five or fewer available slots. It helps monitor package availability and plan suitable promotional or management activities.

```
SQL> SELECT PACKAGE_ID,
  2         PACKAGE_NAME,
  3         AVAILABLE_SLOTS
  4  FROM TRAVELPACKAGE
  5  WHERE AVAILABLE_SLOTS <= 5
  6  ORDER BY AVAILABLE_SLOTS;

no rows selected
```

6. PERFORM CATEGORY-WISE ANALYSIS

Category-wise analysis helps understand the distribution and performance of different travel package categories. The CATEGORY_ID column in the TRAVELPACKAGE table is used to group packages according to their categories.

1. Total number of packages in each category

This analysis calculates the total number of travel packages available in each category using the COUNT() function. It helps identify categories containing more packages.

```
SQL> SELECT CATEGORY_ID,
  2         COUNT(*) AS TOTAL_PACKAGES
  3  FROM TRAVELPACKAGE
  4  GROUP BY CATEGORY_ID
  5  ORDER BY TOTAL_PACKAGES DESC;

CATEGORY_ID TOTAL_PACKAGES
----------- --------------
          2              2
          6              2
          3              2
          4              2
          5              2
          1              1

6 rows selected.
```

2. Average package price in each category

This analysis calculates the average price of travel packages in each category using the AVG() function. It helps compare package prices across different categories.

```
SQL> SELECT CATEGORY_ID,
  2         AVG(PRICE) AS AVERAGE_PRICE
  3  FROM TRAVELPACKAGE
  4  GROUP BY CATEGORY_ID
  5  ORDER BY AVERAGE_PRICE DESC;

CATEGORY_ID AVERAGE_PRICE
----------- -------------
          2         60000
          4         37500
          3         23500
          5         21500
          1         13000
          6         11500

6 rows selected.
```

3. Highest-priced category

This analysis identifies the category containing the highest-priced travel package. The maximum package price of each category is calculated and compared.

```
SQL> SELECT CATEGORY_ID,
  2         MAX(PRICE) AS HIGHEST_PRICE
  3  FROM TRAVELPACKAGE
  4  GROUP BY CATEGORY_ID
  5  ORDER BY HIGHEST_PRICE DESC;

CATEGORY_ID HIGHEST_PRICE
----------- -------------
          2         65000
          4         45000
          5         28000
          3         25000
          6         14000
          1         13000

6 rows selected.
```

4. Average customer rating by package category

This analysis calculates the average customer rating for each travel package category. The TRAVELPACKAGE and REVIEW tables are joined using PACKAGE_ID. It helps understand customer satisfaction across different package categories.

```
SQL> SELECT T.CATEGORY_ID,
  2         AVG(R.RATING) AS AVERAGE_RATING,
  3         COUNT(R.REVIEW_ID) AS TOTAL_REVIEWS
  4  FROM TRAVELPACKAGE T
  5  LEFT JOIN REVIEW R
  6  ON T.PACKAGE_ID = R.PACKAGE_ID
  7  GROUP BY T.CATEGORY_ID
  8  ORDER BY AVERAGE_RATING DESC;

CATEGORY_ID AVERAGE_RATING TOTAL_REVIEWS
----------- -------------- -------------
          6                            0
          5                            0
          1              5             1
          3              5             1
          2            3.5             2
          4              2             1

6 rows selected.
```

7. GENERATE BUSINESS ANALYTICS REPORTS

Report 1: Booking Performance Report

This report includes:

- Total booking amount
- Number of bookings
- Average booking amount
- Highest booking amount

The BOOKING table is used to generate the overall booking performance report. Aggregate functions are used to calculate the total amount, number of bookings, average amount, and highest booking amount. This report helps evaluate the overall financial performance of the system.

```
SQL> SELECT COUNT(*) AS TOTAL_BOOKINGS,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE,
  3         AVG(TOTAL_AMOUNT) AS AVERAGE_AMOUNT,
  4         MAX(TOTAL_AMOUNT) AS HIGHEST_AMOUNT
  5  FROM BOOKING;

TOTAL_BOOKINGS TOTAL_REVENUE AVERAGE_AMOUNT HIGHEST_AMOUNT
-------------- ------------- -------------- --------------
             8        335000          41875          65000
```

Report 2: Customer Analytics Report

This report includes:

- Customer name
- Number of bookings
- Total spending
- Booking frequency

The CUSTOMER and BOOKING tables are joined to generate the customer analytics report. The number of bookings and total spending of each customer are calculated. Customers are classified according to their booking frequency. Customers who have not placed any bookings are also included in the report

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS,
  5         NVL(SUM(B.TOTAL_AMOUNT), 0) AS TOTAL_SPENDING,
  6         CASE
  7             WHEN COUNT(B.BOOKING_ID) = 0
  8                 THEN 'NO BOOKINGS'
  9             WHEN COUNT(B.BOOKING_ID) = 1
 10                 THEN 'ONE BOOKING'
 11             ELSE 'FREQUENT CUSTOMER'
 12         END AS BOOKING_FREQUENCY
 13  FROM CUSTOMER C
 14  LEFT JOIN BOOKING B
 15  ON C.CUSTOMER_ID = B.CUSTOMER_ID
 16  GROUP BY C.CUSTOMER_ID,
 17           C.FIRST_NAME,
 18           C.LAST_NAME
 19  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_BOOKING
S
-------------------- -------------------- -------------------- -------------
-
TOTAL_SPENDING BOOKING_FREQUENCY
-------------- ----------------
C004                 Divya                Suresh
1
         65000 ONE BOOKING

C008                 Ananya               Krishnan
1
         55000 ONE BOOKING

C003                 Rahul                Raj
1
         55000 ONE BOOKING
```

```
CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_BOOKING
S
-------------------- -------------------- -------------------- -------------
-
TOTAL_SPENDING BOOKING_FREQUENCY
-------------- ----------------
C007                 Vishnu               Kumar
1
         45000 ONE BOOKING

C002                 Priya                Sharma
1
         36000 ONE BOOKING

C006                 Sneha                Ravi
1
         30000 ONE BOOKING


CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_BOOKING
S
-------------------- -------------------- -------------------- -------------
-
TOTAL_SPENDING BOOKING_FREQUENCY
-------------- ----------------
C005                 Karthik              Mohan
1
         25000 ONE BOOKING

C001                 Arun                 Kumar
1
         24000 ONE BOOKING

C010                 Meena                Devi
0
             0 NO BOOKINGS
```

```
CUSTOMER_ID          FIRST_NAME           LAST_NAME            TOTAL_BOOKING
S
-------------------- -------------------- -------------------- -------------
-
TOTAL_SPENDING BOOKING_FREQUENCY
-------------- ----------------
C009                 Naveen               Raj
0
             0 NO BOOKINGS


10 rows selected.
```

Report 3: Payment Analytics Report

This report includes:

- Payment mode
- Payment status

- Number of transactions
- Total transaction amount

The PAYMENT table is used to generate the payment analytics report. The transactions are grouped according to payment mode and payment status. The total number of transactions and transaction amounts are calculated to understand payment activity.

```
SQL> SELECT PAYMENT_MODE,
  2         PAYMENT_STATUS,
  3         COUNT(PAYMENT_ID) AS TOTAL_TRANSACTIONS,
  4         SUM(TRANSACTION_AMOUNT) AS TOTAL_AMOUNT
  5  FROM PAYMENT
  6  GROUP BY PAYMENT_MODE, PAYMENT_STATUS
  7  ORDER BY PAYMENT_MODE;

PAYMENT_MODE         PAYMENT_STATUS       TOTAL_TRANSACTIONS TOTAL_AMOUNT
-------------------- -------------------- ------------------ ------------
Card                 Success                               2        61000
Cash                 Pending                               1        30000
Net Banking          Pending                               1        55000
UPI                  Failed                                1        55000
UPI                  Success                               3       134000
```

Report 4: Package and Review Analysis Report

This report includes:

- Package ID
- Package name
- Category ID
- Package price
- Total number of reviews
- Average customer rating

The TRAVELPACKAGE and REVIEW tables are joined to generate the package and review analysis report. The total number of reviews and average rating for each package are calculated. This report helps identify packages with better customer feedback.

```
SQL> SELECT T.PACKAGE_ID,
  2         T.PACKAGE_NAME,
  3         T.CATEGORY_ID,
  4         T.PRICE,
  5         COUNT(R.REVIEW_ID) AS TOTAL_REVIEWS,
  6         NVL(AVG(R.RATING), 0) AS AVERAGE_RATING
  7  FROM TRAVELPACKAGE T
  8  LEFT JOIN REVIEW R
  9  ON T.PACKAGE_ID = R.PACKAGE_ID
 10  GROUP BY T.PACKAGE_ID,
 11           T.PACKAGE_NAME,
 12           T.CATEGORY_ID,
 13           T.PRICE
 14  ORDER BY AVERAGE_RATING DESC;

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE TOTAL_REVIEWS AVERAGE_RATING
----------- ---------- ------------- --------------
       105
Manali Adventure Trip
          3      25000             1              5

       101
Ooty Holiday Package
          1      13000             1              5

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE TOTAL_REVIEWS AVERAGE_RATING
----------- ---------- ------------- --------------

       103
Dubai Holiday Package
          2      55000             1              4

       104
Singapore Tour Package
```

```
PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
CATEGORY_ID      PRICE TOTAL_REVIEWS AVERAGE_RATING
----------- ---------- ------------- --------------
          2      65000             1              3

       107
Goa Honeymoon Package
          4      30000             1              2

       108

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
CATEGORY_ID      PRICE TOTAL_REVIEWS AVERAGE_RATING
----------- ---------- ------------- --------------
Kashmir Honeymoon Package
          4      45000             0              0

       111
Tirupati Pilgrimage Tour
          6       9000             0              0


PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
CATEGORY_ID      PRICE TOTAL_REVIEWS AVERAGE_RATING
----------- ---------- ------------- --------------
       106
Rishikesh Adventure Tour
          3      22000             0              0

       109
Mysore Family Package
          5      15000             0              0
```

```
PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE TOTAL_REVIEWS AVERAGE_RATING
---------- ---------- ------------- --------------

       112
Varanasi Spiritual Tour
          6      14000             0              0

       110
Rajasthan Family Tour

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE TOTAL_REVIEWS AVERAGE_RATING
---------- ---------- ------------- --------------
          5      28000             0              0


11 rows selected.
```

RESULT

Thus, the Week 9 Sales and Customer Analytics System was successfully implemented for the Travel Booking Management System using SQL aggregate functions, joins, subqueries, and grouping operations.

The analysis was used to calculate total booking amounts, average booking values, highest and lowest booking amounts, customer spending, booking frequency, and payment transactions. Package availability and category-wise customer review analysis were also performed.

CONCLUSION

The Week 9 Sales and Customer Analytics System helped analyze booking information and customer behaviour in the Travel Booking Management System.

The reports generated using SQL aggregate functions and grouping operations helped identify frequent customers, high-value customers, payment trends, package availability, and customer ratings.

These reports help understand customer booking behaviour, evaluate system performance, and support better business decisions.